Project Proposal: Multi-Agent Automated Data Analysis Workflow

Executive Summary

This proposal outlines the architecture, implementation, and capabilities of a **Multi-Agent Automated Data Analysis System** built using **LangGraph**, **LangChain**, and **OpenAI (`gpt-4o-mini`)**. The system automates end-to-end data processing workflows—from natural language query parsing to plan generation, secure Python/Pandas code generation, safety auditing, and sandboxed code execution.

By decoupling concerns across specialized agents and enforcing structured safety guardrails with reflection loops, the system ensures that automatically generated code is both analytically sound and secure against malicious or system-disruptive execution.

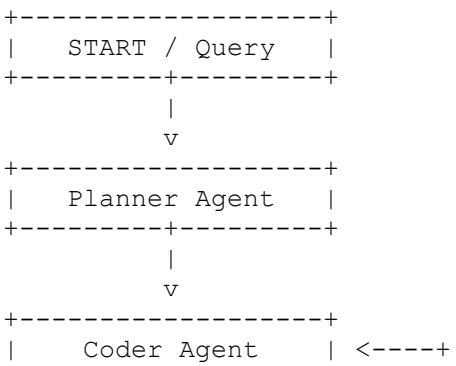
Key Features & System Architecture

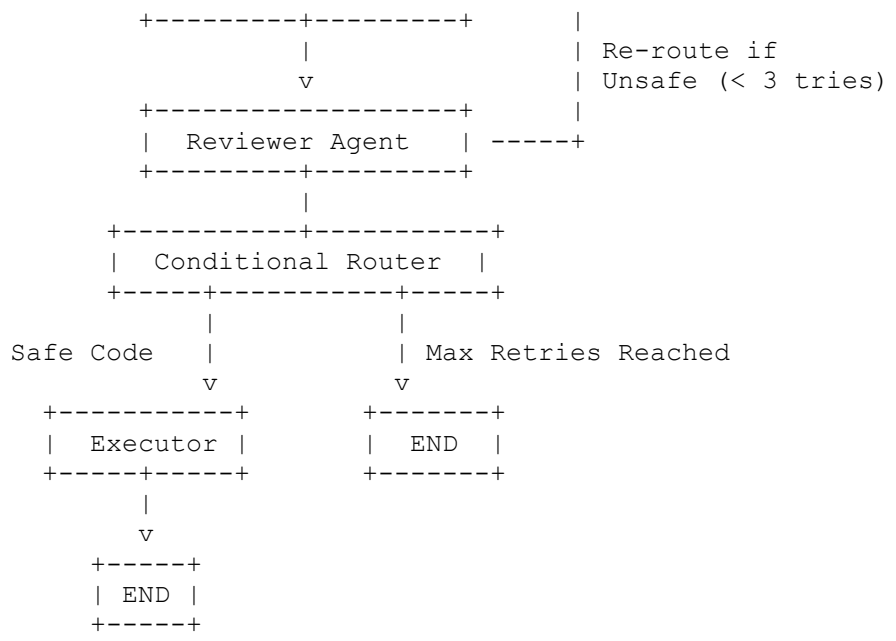
1. Global State Management

The system operates around a shared state (`AgentState`) tracking key workflow data:

- `query`: Natural language request from the user.
- `plan`: Step-by-step data transformation strategy created by the Planner.
- `code`: Generated Python/Pandas code block.
- `review_feedback`: Safety audit results and suggestions.
- `is_safe`: Boolean compliance flag determining workflow progression.
- `iterations`: Retry counter to prevent infinite revision loops (capped at 3).
- `execution_output` & `execution_error`: Results or traceback outputs from sandboxed REPL execution.
- `chart_path`: File path to rendered visualizations.

2. Multi-Agent Network Design





The system comprises four primary execution nodes:

1. **Planner Agent (`planner_node`)**:
    - **Role**: Breaks down natural language questions into structured step-by-step procedures for data cleaning, transformation, and aggregation.
    - **Output**: Detailed task breakdown.
2. **Coder Agent (`coder_node`)**:
    - **Role**: Translates plans and feedback into executable Python code using Pandas and Matplotlib.
    - **Rules & Constraints**:
        - Enforces explicit UTF-8 encoding across file operations for cross-platform stability.
        - Directs charts to be saved locally (`plt.savefig('chart.png')`) instead of using interactive display modes (`plt.show()`).
        - Mandates top-N data limiting (`head(10)`, `nlargest(10, ...)`) on visualizations to ensure uncluttered axis rendering.
        - Prohibits restricted module imports (e.g., `os`, `subprocess`, `requests`).
3. **Reviewer Agent (`reviewer_node`)**:
    - **Role**: Performs security and code compliance audits prior to execution.
    - **Structure**: Returns structured pydantic output (`SafetyReview`) containing `is_safe` boolean flags and explanatory feedback.
    - **Allowed Operations**: Standard Pandas, NumPy, Matplotlib, basic built-ins, and local CSV/PNG operations with explicit UTF-8 encoding.
    - **Disallowed Operations**: OS commands (`os.system`), process spawning (`subprocess`), network calls (`requests`, `socket`), or arbitrary evaluation (`eval`, `exec`).
4. **Executor Agent (`executor_node`)**:
    - **Role**: Executes verified safe code using LangChain's `PythonREPL` sandbox.

    - **Outputs**: Captures standard output, tracebacks/errors, and detects generated image artifacts dynamically.

3. Iterative Refinement & Fallbacks

- **Reflection Loop**: If the Reviewer flags safety violations or bad patterns, the conditional edge (`should_continue`) routes feedback back to the Coder for correction.
- **Retry Safety**: To prevent continuous loops, the workflow halts if code fails safety checks after 3 iterations.

Technology Stack

Component	Technology / Library
Orchestration Framework	LangGraph (`StateGraph`)
Language Model	OpenAI `gpt-4o-mini` via LangChain ChatOpenAI
Execution Environment	`langchain_experimental.utilities.PythonREPL`
Data Processing & Plotting	Pandas, Matplotlib
Environment Management	`python-dotenv`, `pydantic`

Deliverables & Usage

Setup & Requirements

1. Environment file containing `OPENAI_API_KEY`.
2. Python dependencies: `langchain`, `langgraph`, `langchain-openai`, `langchain-experimental`, `pandas`, `matplotlib`, `pydantic`, `python-dotenv`.

Sample Execution Stream

When initialized with a query such as:

"Load the file sales_data.csv, filter rows where Sales > 1000, and calculate the mean revenue by region."

The system completes the full loop:

1. **Planner**: Creates tabular processing tasks.
2. **Coder**: Generates Python/Pandas code adhering to formatting and encoding standard rules.
3. **Reviewer**: Evaluates code against security policies using structured JSON schemas.
4. **Executor**: Runs approved code in the Python REPL and returns dataset summaries or generated chart paths.